

Saankain — File Structure

This document defines the **official repository structure** for **Saankain**, a natural-language restaurant discovery system built by **Nyebe Creations**.

The repository structure is designed to enforce:

```
clear architecture
separation of concerns
predictable module ownership
AI-safe code generation
```

The structure is optimized for:

```
Next.js App Router
TypeScript
AI-assisted development
```

Root Structure

```
/
├─ app
├─ providers
├─ components
├─ hooks
├─ services
├─ lib
├─ types
├─ tests
├─ instructions
├─ docs
├─ public
├─ copilot-instructions.md
└─ package.json
```

/app

Next.js **App Router** entry points.

Contains:

```
pages
layouts
API routes
```

Structure:

```
/app
├── layout.tsx
├── page.tsx
└── api
    └── execute
        └── route.ts
```

Responsibilities:

File	Purpose
layout.tsx	global layout
page.tsx	search UI
route.ts	backend API endpoint

Important rule:

```
No business logic may exist in /app
```

API routes orchestrate modules but do not implement logic.

/components

Reusable **UI components**.

Structure:

```
/components
├── SearchForm.tsx
├── ResultsList.tsx
├── ResultCard.tsx
├── LoadingState.tsx
└── ErrorState.tsx
```

Responsibilities:

```
UI rendering
layout
display state
user interaction events
```

Components must not:

```
call APIs
parse natural language
rank results
perform business logic
```

/hooks

Frontend **logic layer**.

Hooks manage state and interaction logic between UI and data services.

Structure:

```
/hooks
└─ useSearch.ts
```

Responsibilities:

```
search state
loading state
error state
triggering search
storing results
```

Hooks may call **services** but must not call backend modules directly.

/services

Frontend **data access layer**.

Services communicate with backend API endpoints.

Structure:

```
/services
└─ search.ts
```

Responsibilities:

```
call /api/v1/execute
build request parameters
handle network errors
return typed responses
```

Services must not contain UI logic or application state.

/lib

Backend **business logic modules**.

This directory contains the core application logic.

Structure:

```
/lib
├─ parser
│   ├── parseMessage.ts
│   └─ rules.ts
├─ foursquare
│   ├── client.ts
│   ├── searchPlaces.ts
│   └─ transform.ts
├─ ranking
│   └─ rankResults.ts
├─ validation
│   └─ validateExecuteQuery.ts
└─ utils
    └─ logger.ts
```

Parser Module

Location:

```
/lib/parser
```

Purpose:

Convert natural language input into structured search parameters.

Example:

```
cheap sushi near makati open now
```

Parsed result:

```
{  
  "cuisine": "sushi",  
  "locationText": "makati",  
  "priceLevel": 1,  
  "openNow": true  
}
```

Foursquare Module

Location:

```
/lib/foursquare
```

Responsibilities:

```
build API request  
authenticate  
call Foursquare API  
normalize results
```

External API responses must never be returned directly.

Ranking Module

Location:

```
/lib/ranking
```

Purpose:

Improve search relevance.

Signals include:

```
cuisine match  
price match  
open status  
rating preference  
distance
```

Ranking is deterministic.

Validation Module

Location:

```
/lib/validation
```

Responsibilities:

```
validate API parameters  
verify code gate  
validate message input
```

Validation must occur before parsing.

/types

Centralized **TypeScript** type definitions.

Structure:

```
/types
├── search.ts
├── restaurant.ts
└── api.ts
```

Examples:

```
ParsedSearch
RestaurantResult
ExecuteResponse
```

All shared types must live here.

/tests

Automated tests.

Structure:

```
/tests
├── parser
│   └── parseMessage.test.ts
├── ranking
│   └── rankResults.test.ts
├── api
│   └── execute.test.ts
└── foursquare
    └── transform.test.ts
```

Tests must cover:

```
parser logic
ranking logic
API validation
response normalization
```

External APIs must be mocked.

/instructions

AI instruction files used by **Copilot**.

Structure:

```
/instructions
├─ parser.instructions.md
├─ foursquare.instructions.md
├─ api.instructions.md
├─ ranking.instructions.md
├─ pages.instructions.md
└─ types.instructions.md
```

These files guide AI-generated code to respect repository architecture.

/docs

Developer documentation.

Structure:

```
/docs
├─ architecture.md
├─ api-contract.md
├─ file-structure.md
└─ parser-design.md
```

These documents describe:

```
system design
API behavior
architecture decisions
```

/public

Static assets.

Structure:


```
/public
```

Contains:

```
images  
icons  
logos
```

No application logic.

Import Rules

Use root imports.

Correct:

```
import { parseMessage } from "@lib/parser/parseMessage"
```

Incorrect:

```
import { parseMessage } from "../../lib/parser/parseMessage"
```

Root imports improve readability and prevent fragile paths.

Architecture Boundaries

These rules must never be violated.

Rule 1 — UI isolation

```
UI cannot contain business logic
```

Rule 2 — Logic isolation

Hooks manage state but cannot call backend modules.

Rule 3 — API orchestration

API routes coordinate modules but do not implement logic.

Rule 4 — External services isolation

All third-party API calls must exist only in:

```
/lib/foursquare
```

Golden Rule

If new code does not clearly belong to a directory:

```
stop and ask before creating it
```

Architecture must not evolve accidentally.